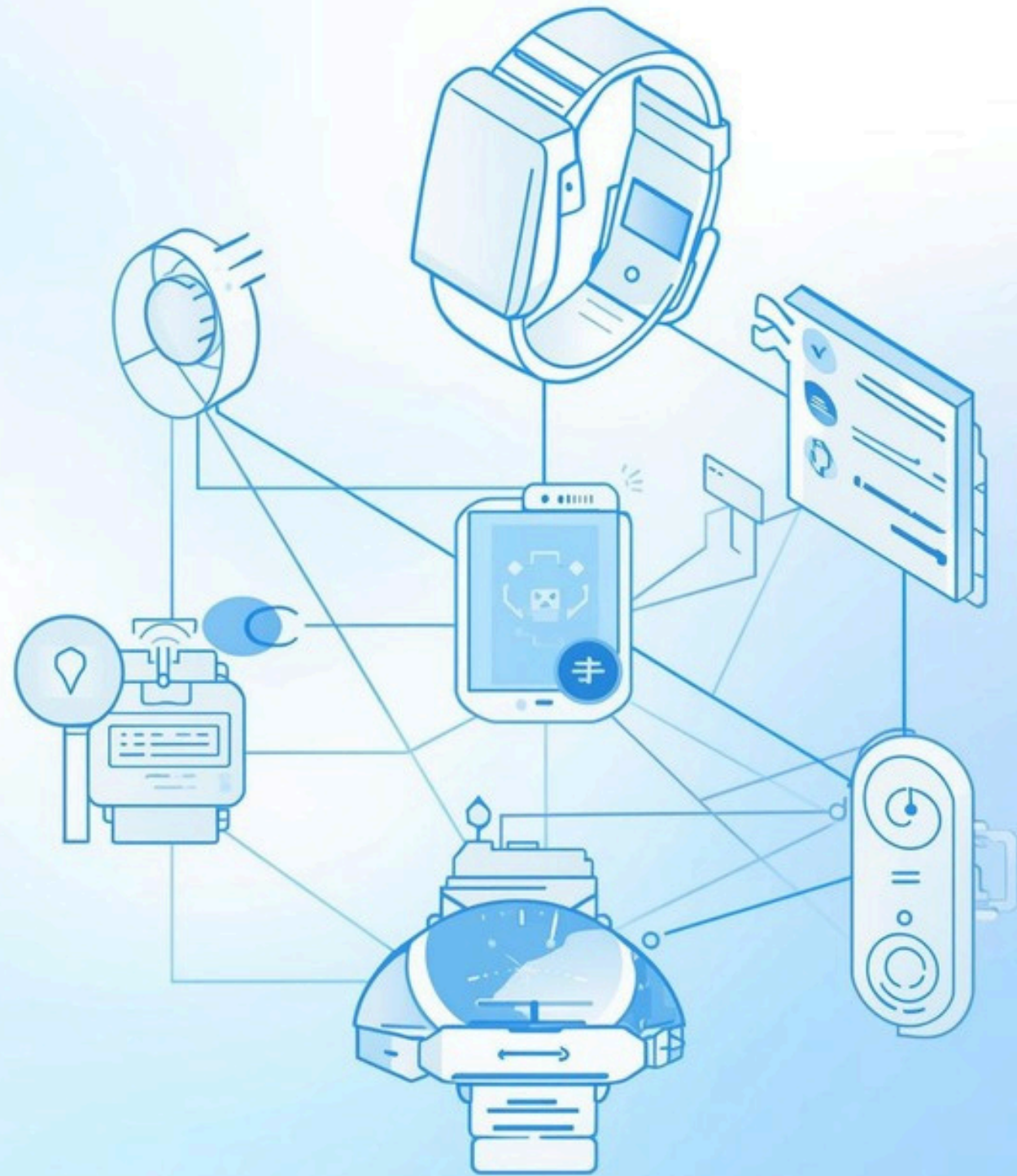


Lab 3: Introduction to BLE

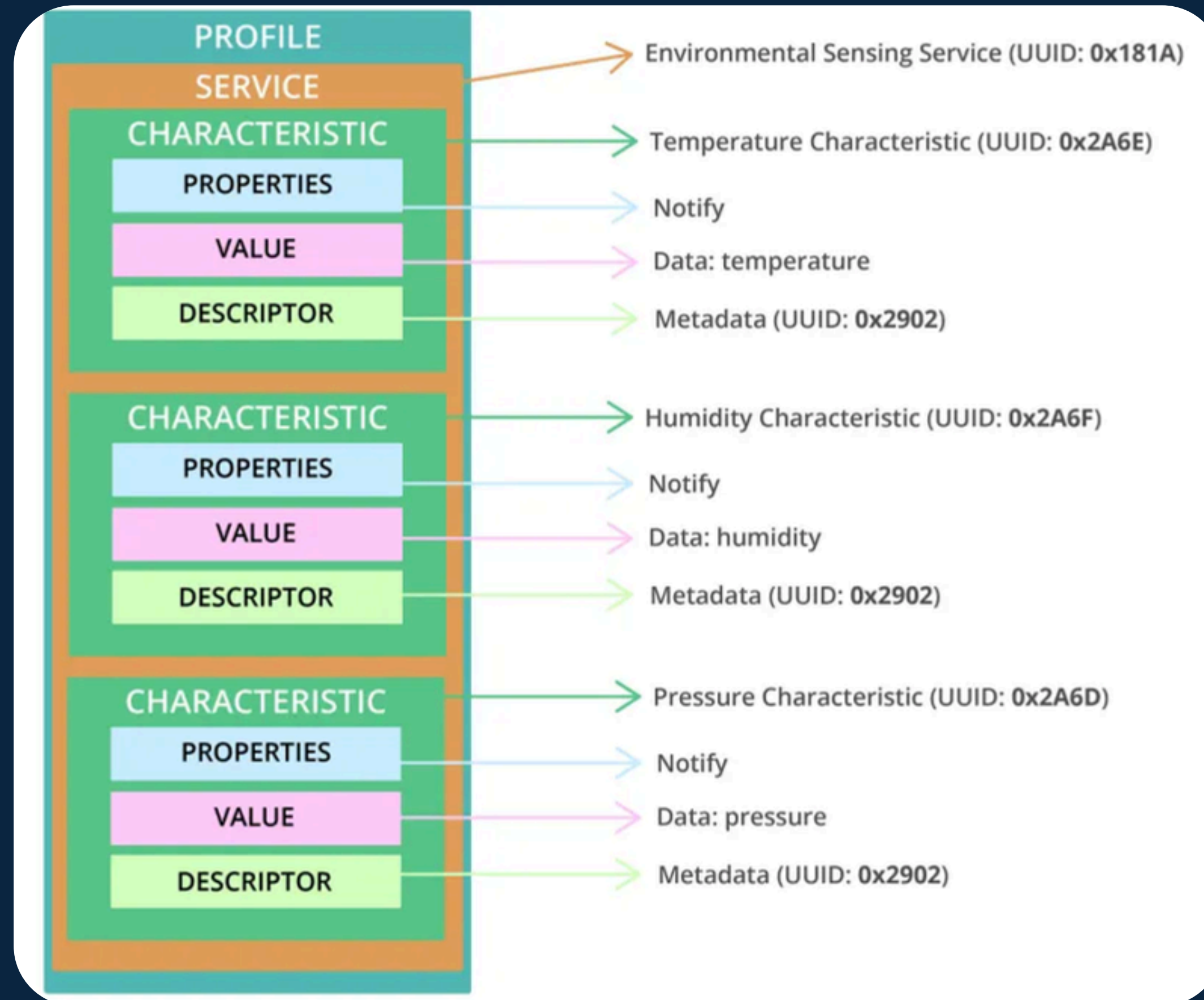
Bluetooth Low Energy (BLE)



Andrea Novero, Filippo Bragato

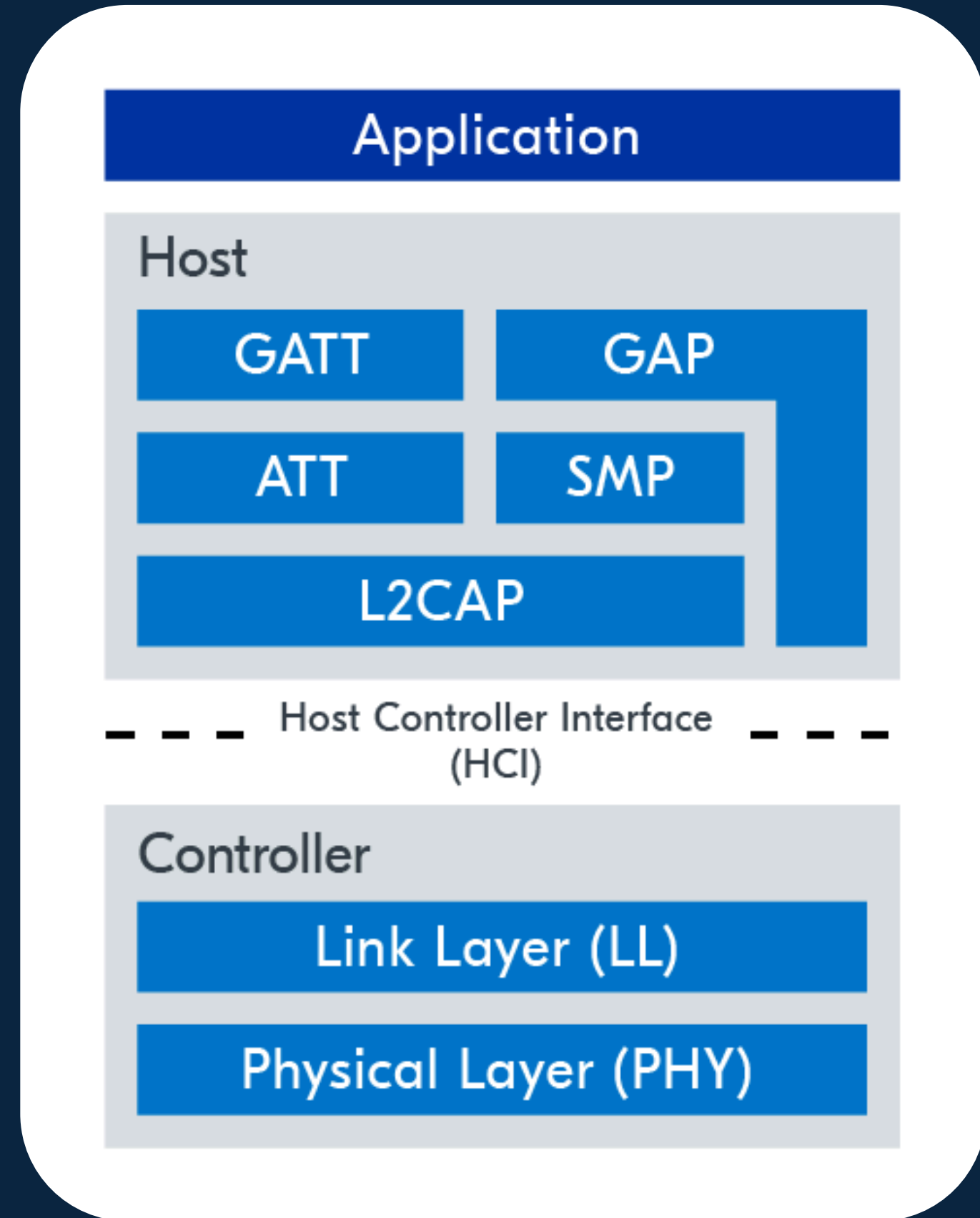
BLE – Quick guide

- **Built for low-power IoT:**
wearables, sensors, beacons
- **Server** : exposes
 - Services
 - Characteristics
 - Values
- **Client** operations:
 - scan,
 - connect
 - discover services
 - **read/write** values



BLE – Stack

- **Host (software):**
GAP, GATT/ATT, L2CAP, SMP
- **Controller (hardware/firmware):**
Link Layer (LL) + PHY (radio)
- **HCI = Host Controller Interface:**
The command/event channel between Host and Controller, HCI commands are sent by the Host and executed inside the Bluetooth controller



BLE Standard: GATT Services' UUIDs

Bluetooth SIG (Special Interest Group): defines the standard <https://www.bluetooth.com/>

Assigned **UUID** (Universally Unique Identifier) **services, characteristics**
<https://www.bluetooth.com/specifications/assigned-numbers/>

E.g., **GATT layer services** (up to 255):

3.4 GATT Services

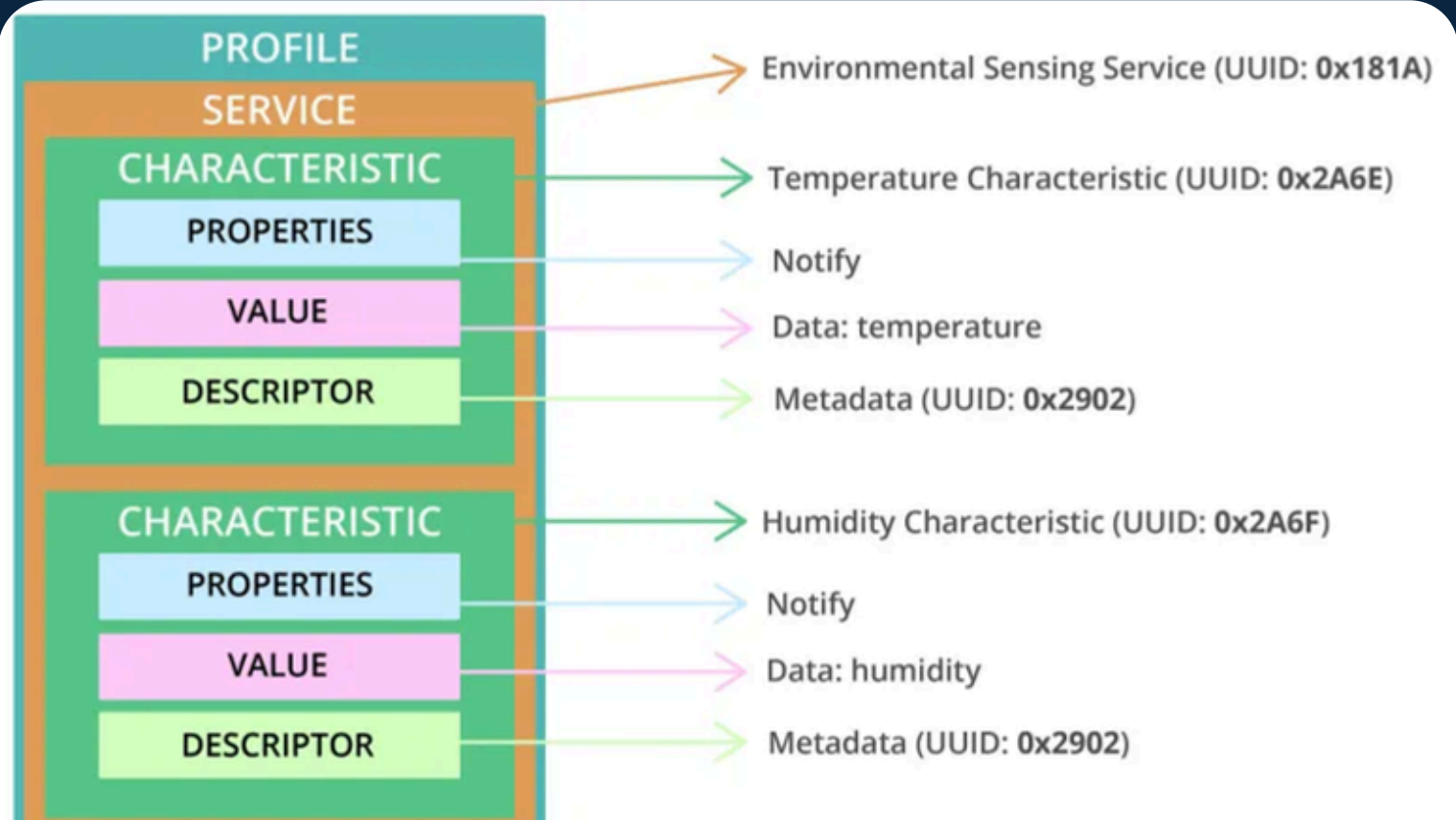
See Bluetooth Core Specification [Vol 3] Part G, Section 3.3.1 [4].

3.4.1 Services by Name

Last Modified: 2025-11-04

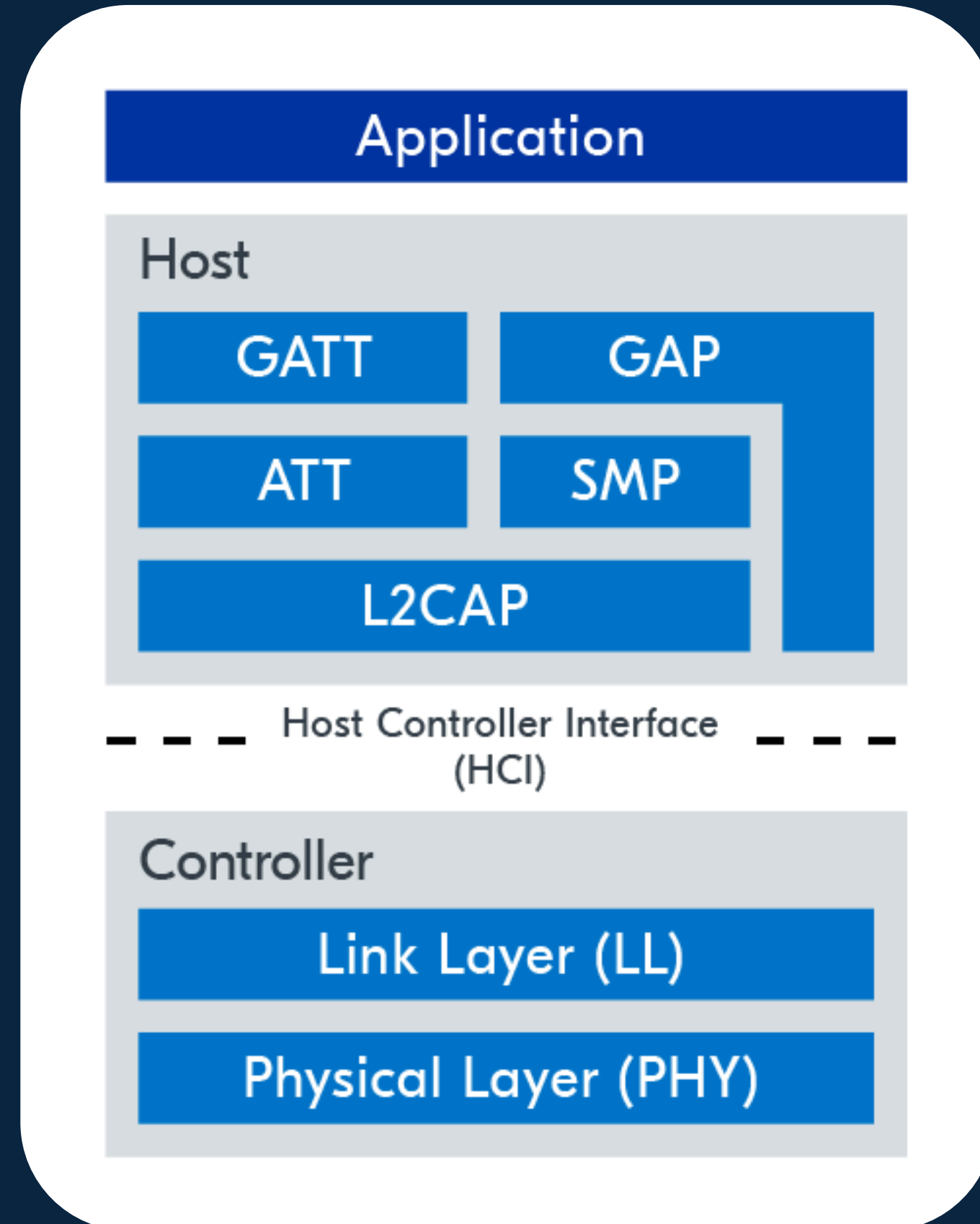
Filename: [assigned_numbers/uuids/service_uuids.yaml](#)

Name	UUID
Alert Notification Service	0x1811
Audio Input Control Service	0x1843
Audio Stream Control Service	0x184E



BLE – Workflow

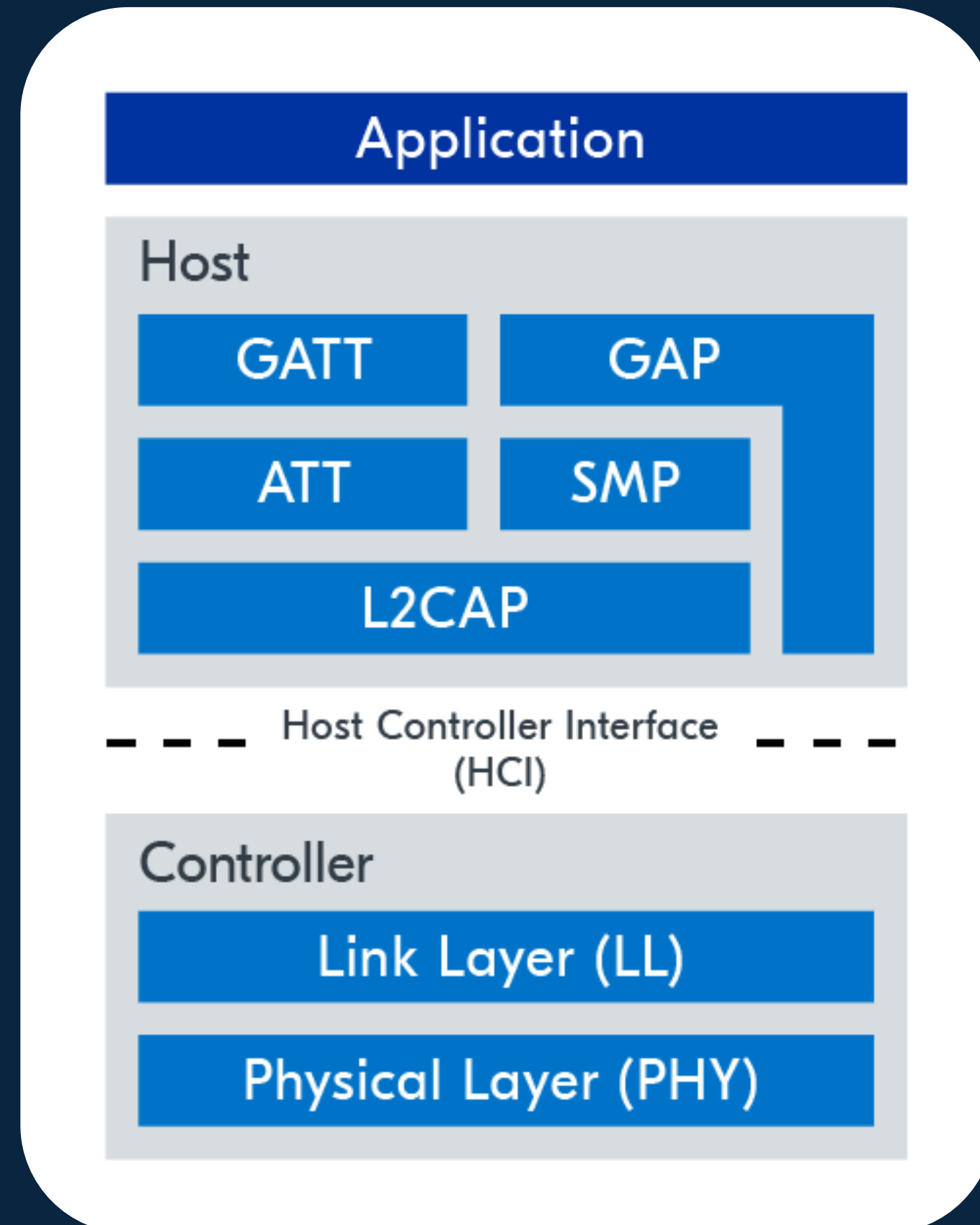
1. **GAP** decides action (advertise / scan / connect)
2. Host sends **HCI command** to the Controller
3. Controller runs **Link Layer/PHY actions**
4. Once **GAP connected** (GATT, ATT, L2CAP layers):
 - **GATT layer**:
 - handles **connection** and **BLE profiles** (services/characteristics)
 - **L2CAP** handles flow control
 - **ATT messages** (Read/Write/Exchange MTU)
 - **SMP**: security



BLE – Lab exercise

What We Will Observe

- **GAP layer operations:**
 - Server: advertise
 - Client: scanning, connecting
- **GATT + ATT layers**
 - **Services and characteristics** on the server
 - **ATT messages:** Exchange MTU, Read By Group Type, Read
- **HCI commands** between host and controller; e.g.,
 - LE Set Advertising Parameters
 - LE Set Scan Enable
 - LE Create Connection



BLE – Lab Scenario: Smartwatch

- **Smartwatch (Peripheral, GATT Server)**

- Exposes services:

- Device Information Service (0x180A)

- Characteristic: Manufacturer Name → e.g., "Mi Band"

- Heart Rate Service (0x180D)

- Characteristic: Heart Rate Measurement → e.g., 91 bpm



- **Smartphone (Central, GATT Client)**

- Scans, finds, and connects to the smartwatch

- Discovers services and characteristics

- **Goal: Read device 'name' and 'heart rate' value**



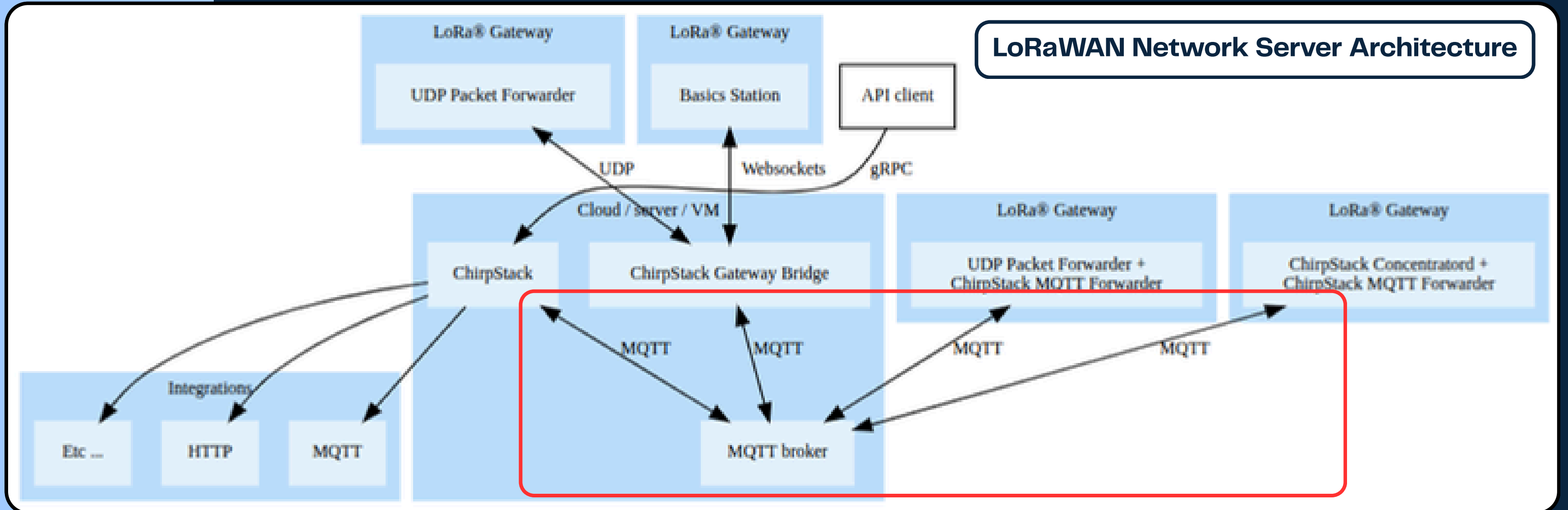
- **You choose each step** (advertise, scan, connect, discover, read) **and watch GAP, HCI, and GATT/ATT messages** in the terminal.

Run BLE simulation

- go to 'examples' folder
 - export PYTHONPATH=..
 - pip3 install typing_extensions pyee
 - (if "no module XXX" => pip3 install XXX)
 - run: python3 ble_interactive_lab.py
- **Goal: Read BLE device 'name' and 'heart rate' values**



Message Queuing Telemetry Transport



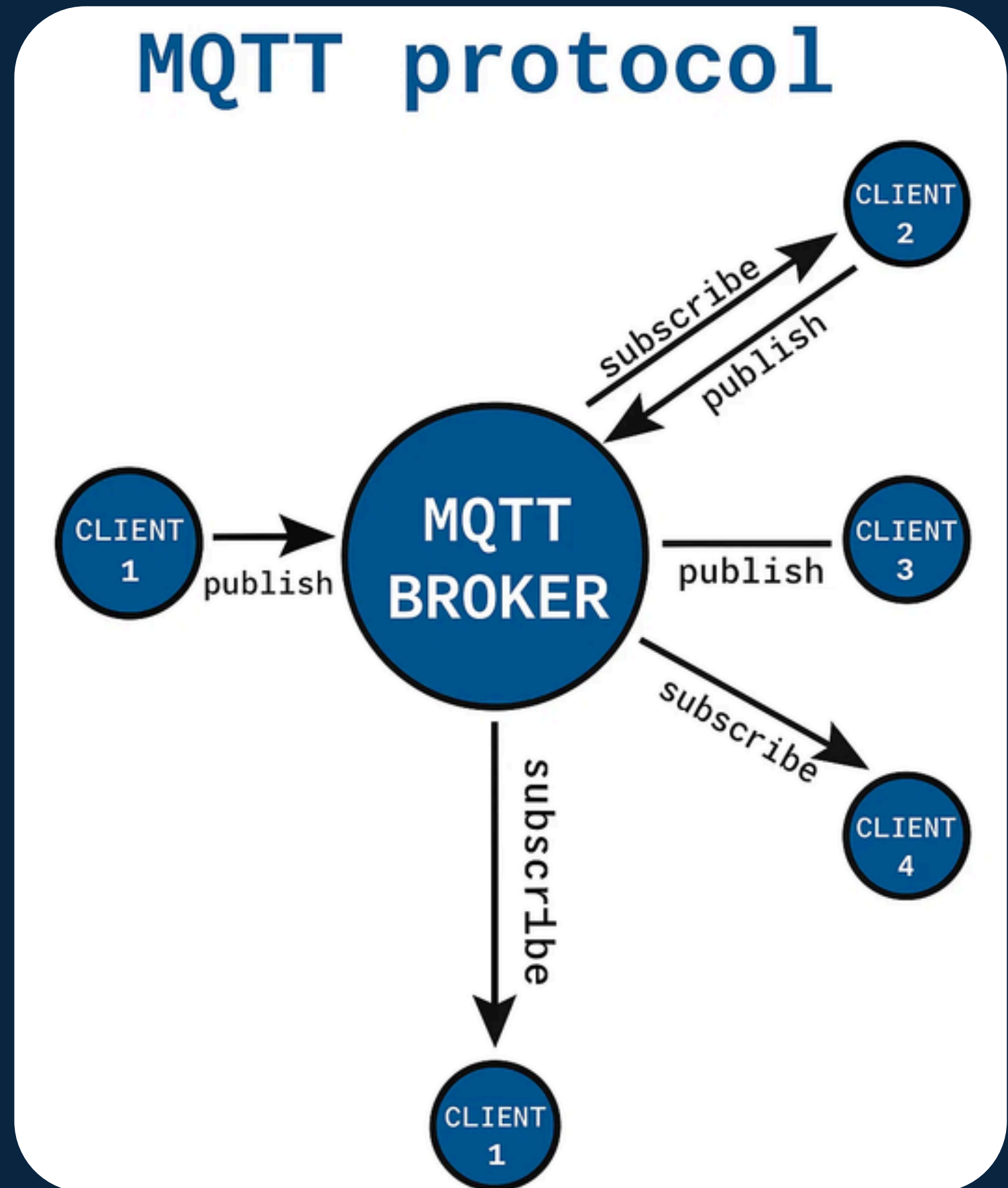
MQTT Protocol

MQTT roles:

- One MQTT Broker (Server)
- Many MQTT Clients

How it works:

- Publish/Subscribe paradigm
- 3 QoS (Quality Of Service) levels



MQTT Protocol

MQTT Messages

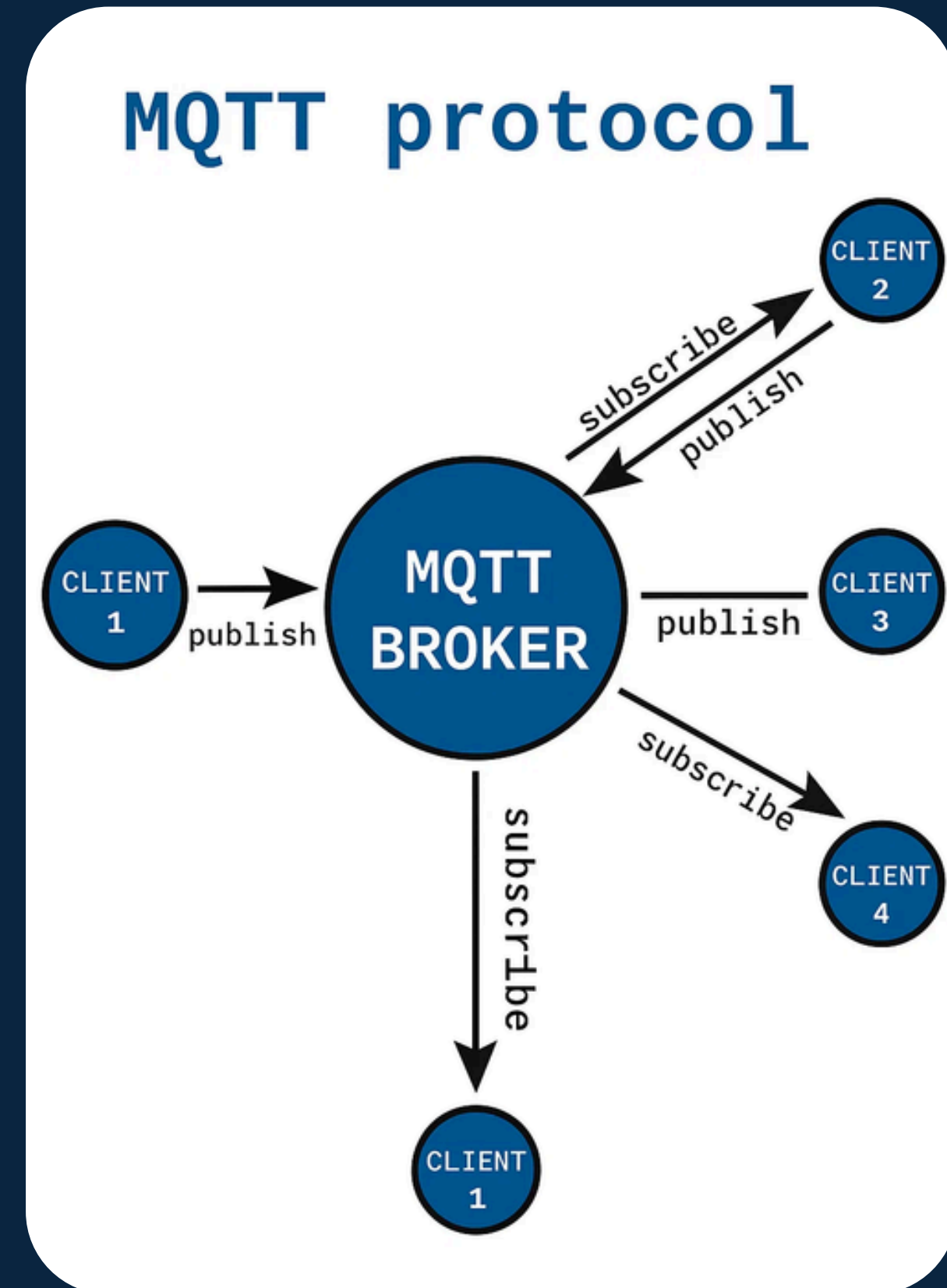
- **Published** by **Clients** and **routed** by **Broker**
- Identified by a **topic**
- Have a QoS level (0, 1, or 2)

MQTT Clients

- **Publish** messages on **topics**
- **Subscribe** to topics
- Choose a **QoS** level (0, 1, or 2) for **each publish/subscription**

MQTT Broker

- **Receives** client **subscriptions**
- **Receives** published **messages**
- **Routes messages** to clients subscribed to the matching topic



MQTT QoS levels

QoS 0 – At most once

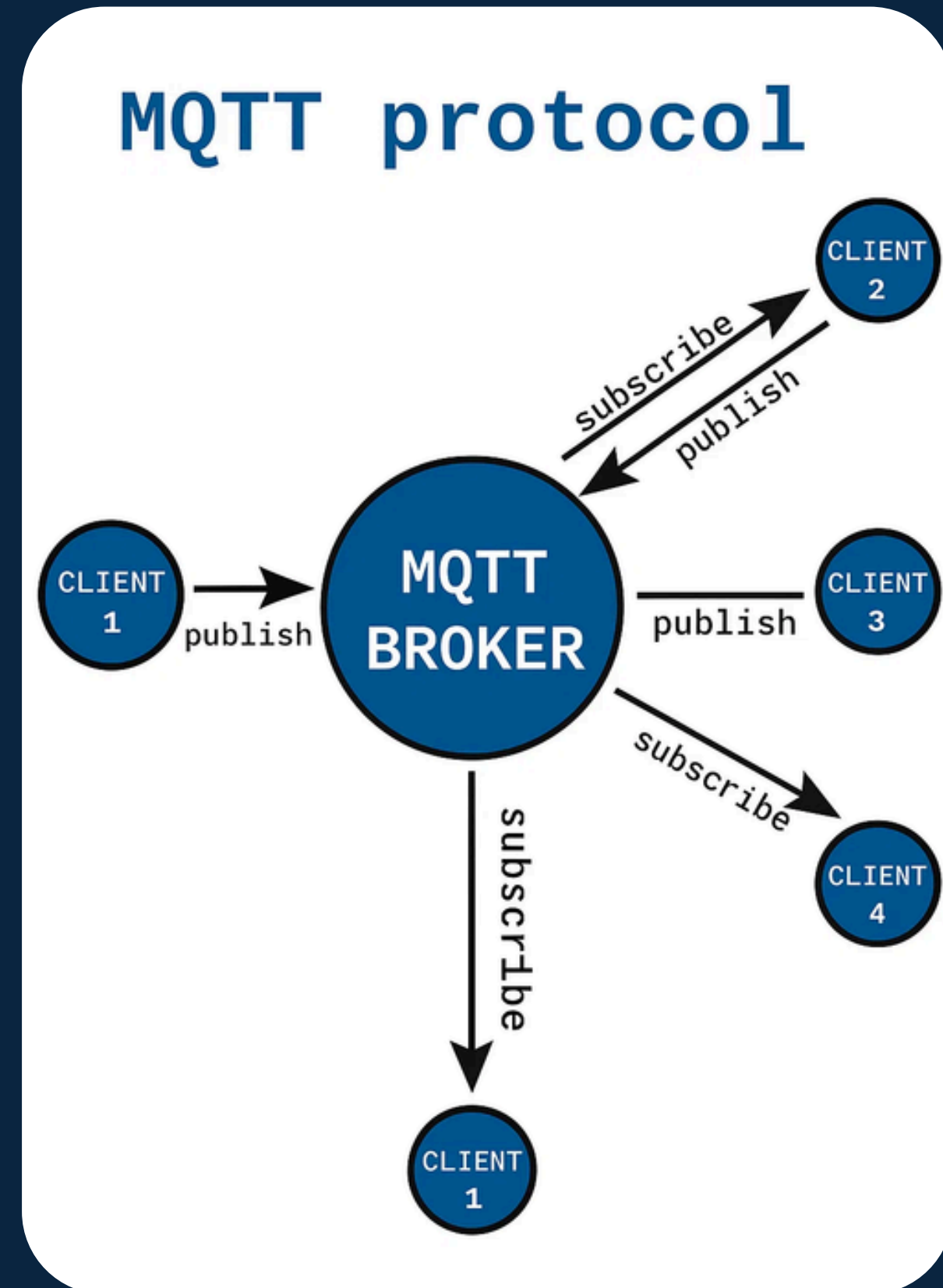
- **Message is sent once**, no ACK, no retransmission

QoS 1 – At least once

- **One ACK (PUBACK)**
- **Retransmissions until ACK**
→ the receiver may get the same message multiple times

QoS 2 – Exactly once

- Guaranteed **single reception**
- Full 4-step **handshake**:
 - PUBLISH
 - PUBREC (Broker confirms reception)
 - PUBREL (Client allows for release/forwarding)
 - PUBCOMP (Broker confirms end of operations)



You will see that **publishers have QoS priority on messages**

MQTT Lab exercise 1

- **Open a MQTT terminal to run a MQTT instance:**

singularity shell docker://eclipse-mosquitto

- **Inside a MQTT instance run the Broker:**

mosquitto -v

- **Inside a MQTT instance subscribe the client** to the topic “test/topic”, **QoS 0** and client name **sub_q0**:

mosquitto_sub -h 127.0.0.1 -p 1883 -t 'test/topic' -v -q 0 -i sub_q0

- **Inside a MQTT instance publish** the message “hello world” to topic “test/topic”, **QoS 0** and client name **pub_q0** (the client is created, it publishes and dies):

mosquitto_pub -h localhost -p 1883 -t "test/topic" -m "hello world" -q 0 -i pub_q0

- **Look at the messages exchanged and the broker logs: SUBSCRIBE, PUBLISH, etc.**
- **Publish and subscribe with different QoS, watch (and try to understand) the broker logs**

MQTT Lab exercise 2

- **Create 1 broker**
- **Create 4 subscribers:**
 - **Sub_a** subscribed to “topic/a”, QoS 1
 - **Sub_b** subscribes to “topic/b”, QoS 2
 - **Sub_n** subscribes to “topic/#”, QoS 0
 - **Sub_m** subscribes to “#”, QoS 0
- **Publish 4 messages with topics:**
 - **topic/a** QoS 1
 - **topic/b** QoS 2
 - **topic/b** QoS 0
 - **dei** QoS 0
- **What happens? What happens when using # on subscribing? What happens to QoS levels?**